

1. 정렬 알고리즘 설명

Bubble Sort : 인접한 두 원소를 비교하여 순서가 맞지 않으면 교환하는 과정을 반복함으로써, 가장 큰 원소를 배열의 끝으로 이동시킨다. 배열의 첫 번째 원소부터 시작하여 인접한 쌍을 비교한다. 만약 앞선 원소가 뒤의 원소보다 크면 두 원소의 위치를 교환한다. 이 과정을 배열의 끝까지 반복하면 가장 큰 원소는 배열의 맨 끝자리에 위치하게 된다. 두 번째 순회에서는 이미 가장 큰 원소가 끝에 정렬되었으므로, 비교 범위를 1 줄여서 다시 앞의 과정을 반복한다. 배열의 크기를 n 이라 할 때 이 과정을 총 $n-1$ 번 반복하여 정렬을 완료한다.

Insertion Sort : 삽입 정렬은 두 번째 원소($i=1$)부터 시작하여 배열을 순차적으로 확장되는 정렬된 부분 배열과 나머지 부분으로 나누어 처리한다. 현재 인덱스 i 의 원소를 key 로 선택하고, $i-1$ 부터 시작하는 정렬된 부분(j)을 역방향으로 순회하며 탐색한다. 이 탐색 과정에서 $value[j]$ 가 key 보다 클 때마다 $value[j]$ 를 $value[j+1]$ 로 오른쪽으로 한 칸 Shift시킨다. 더 이상 key 보다 큰 원소가 없거나 배열의 시작에 도달하여 이 이동이 멈추면, 비어있는 $value[j+1]$ 위치에 key 를 삽입한다. 이 과정을 배열의 모든 원소에 대해 반복하여 정렬을 완료한다.

Heap Sort : 힙 정렬은 먼저 배열을 Max Heap 구조로 변환하는 Build Max Heap 단계로 시작한다. 이는 배열의 중간 지점($N/2 - 1$)부터 루트 노드(0)까지 역순으로 `build_heap()` 함수를 호출하여 배열 전체를 최대 힙 구조로 만든다. 힙 생성 후, 정렬 단계에서는 배열의 마지막 인덱스($i = N-1$)부터 루트(0)까지 반복 순회한다. 각 반복에서 힙의 루트 원소와 $value[i]$ 를 교환하여 최대값을 배열의 정렬된 위치로 이동시킨다. 이후 힙의 크기를 $i-1$ 로 줄이고, 새로운 루트에서 다시 최대 힙 속성을 복원한다. 이 과정을 통해 최대값을 하나씩 추출하며 배열을 오름차순으로 정렬한다.

Merge Sort : 병합 정렬은 먼저 배열을 중간 지점(mid)을 기준으로 두 개의 서브 배열로 재귀적으로 분할한다. 이 분할 과정은 배열의 크기가 1이 될 때까지 계속된다. 재귀 호출이 반환되면 병합작업을 수행한다. 병합 시 두 개의 정렬된 서브 배열을 생성하여 원소를 복사한 후 두 서브 배열의 포인터를 사용하여 원소를 순차적으로 비교한다. 비교 결과 더 작은 원소를 원본 배열의 해당 위치에 순서대로 복사해 넣는 방식으로 정렬된 하나의 배열로 병합한다.

Quick Sort : 퀵 정렬은 Partition 작업을 중심으로 재귀적으로 수행된다. 먼저 정렬할 부분 배열($low \sim high$)에서 $high$ 원소를 $pivot$ 으로 선택한다. 분할 단계에서는 low 부터 $high-1$ 까지 순회하면서 피벗보다 작은 원소를 왼쪽으로 교환하여 모은다. 순회가 완료되면 피벗을 $value[i+1]$ 위치에 삽입하여 피벗보다 작은 모든 원소는 왼쪽에, 큰 원소는 오른쪽에 오도록 확정하며 피벗의 최종 위치(k)를 반환한다. 이후 반환된 피벗 위치(k)를 기준으로 low 부터 $k-1$, 그리고 $k+1$ 부터 $high$ 까지의 부분 배열에 대해 재귀적으로 퀵 정렬을 호출한다.

Radix Sort : 기수 정렬은 먼저 입력 배열을 순회하여 음수와 양수로 분리한다. 음수는 절댓값으로 변환하여 저장한다. 이후 정렬할 배열(양수 및 음수의 절댓값 배열)에서 가장 큰 값의

자릿수를 확인하고, d번의 반복(1, 10, 100, ... 자릿수(exp)에 대해) 동안 Counting Sort를 호출한다. Counting Sort는 현재 exp 자릿수 값의 빈도를 기록하고, 누적 합을 계산하여 원소의 최종 위치를 파악한다. 원 배열을 역순으로 순회하며 누적 합을 이용하여 원소를 output 배열에 정렬시킨후 다시 원본 배열에 복사한다. 최종적으로 정렬이 완료된 절댓값 배열을 다시 음수로 변환하고 역순으로 배치한 후 양수부분도 마찬가지로 정렬한 후 정렬된 양수 배열을 뒤에 붙여 최종 정렬된 배열을 완성한다.

2. 정렬시간 측정 및 분석

data개수를 500개, 5000개, 50000개로 나누어 실행했다. 각 경우마다 10번을 반복하여 평균값을 내었다. 각 data의 범위는 $(-10^9, 10^9)$ 이다.

data 개수별로 분석한 결과. (평균/최소/최대/표준편차, 단위=ms)

	data 개수=500	data 개수=5000	data 개수=50000
Bubble sort	1.3 / 1.1 / 1.5 / 0.1	28.2 / 26.1/ 30.5 / 1.5	4823.1 / 4600.9/ 5050.2/ 150.1
Insertion sort	0.6 / 0.5 / 0.7 / 0.05	3.3 / 3.0 / 3.6 / 0.2	197.5 / 185.3/ 210.1/ 8.4
Heap Sort	0.1 / 0.08 / 0.12 / 0.01	0.9 / 0.8 / 1.1/ 0.05	5.7 / 5.3/6.0 / 0.1
Merge Sort	0.1 / 0.07 / 0.11 / 0.01	0.8 / 0.7 / 0.9 / 0.05	5.9 / 5.7 / 6.1 / 0.1
Quick Sort	0.2 / 0.16/ 0.25 / 0.03	0.7 / 0.63/ 0.79 / 0.04	6.2 / 5.9 / 6.5 / 0.2
Radix Sort	0.3 / 0.23 / 0.35 / 0.03	4.1 / 3.8 / 4.4 / 0.2	33.7 / 32.0 / 35.5 / 1.2

분석)

- 1) heap sort , merge sort, quick sort는 항상 가장 빠른 알고리즘이다.
- 2) bubble sort는 항상 가장 느린 알고리즘이다.
- 3) radix sort는 bubblesort 보다는 항상 빠르지만, 데이터 개수가 5000개 일땐 insertion sort 보다 느리고 heap sort, merge sort , quick sort 보다는 항상 느렸다. radix sort는 이론적으로 $O(n)$ 이라 가장 빠른 알고리즘이나 앞의 상수 계수가 매우커서(kn 의 k) n 이 매우 크지 않은 이상 데이터 개수가 적을 때는 radix알고리즘보다 빠른 알고리즘이 존재다고 생각할 수 있다.

3.Search의 동작 방식

특수한 경우가 아닌이상 평균적으로 quicksort가 가장 효율적인 알고리즘이므로 기본적으로 quicksort를 사용하도록 한다.

data개수=50000으로 설정후 각 자리수별로 10번씩 반복하여 실행시간의 평균을 정리하면 다음과 같다.

1) 최대자리수에 따른 radix알고리즘의 효율성 분석

(평균/최소/최대/표준편차, 단위=ms)

	최대자리수= 3자리	최대자리수= 4자리	최대자리수= 5자리
bubble sort	4820.7 / 4650.1 / 5010.5 / 110.0	4844.1 / 4680.0 / 5050.2 / 115.0	4819.5 / 4660.0 / 5000.0 / 105.0
insertion sort	243.5 / 230.1 / 260.0 / 10.0	221.1 / 209.0 / 235.5 / 8.0	210.2 / 199.0 / 225.1 / 7.5
heap sort	6.5 / 6.2 / 6.9 / 0.3	6.2 / 5.9 / 6.6 / 0.3	6.5 / 6.2 / 6.9 / 0.3
merge sort	6.1 / 5.8 / 6.5 / 0.2	6.6 / 6.2 / 7.0 / 0.2	6.4 / 6.1 / 6.8 / 0.2
quick sort	5.9 / 5.5 / 6.2 / 0.3	6.9 / 6.4 / 7.4 / 0.3	6.5 / 6.1 / 7.0 / 0.3
radix sort	3.9 / 3.6 / 4.2 / 0.2	4.1 / 3.8 / 4.4 / 0.2	12.2 / 11.1 / 13.5 / 0.8

최대자리수가 5자리일때는 heap,merge,quick sort가 여전히 제일 빨랐으나 자리수가 4자리 이하일 때는 radix sort가 heap, merge, quick sort 보다 유의미 하게 빨랐다. 따라서 sort 알고리즘을 이용할 때 자리수가 4자리 이하의 데이터를 받을 때에는 radix sort를 호출하도록 하였다.

2) 충돌발생율에 따른 알고리즘별 효율성 분석

(평균/최소/최대/표준편차, 단위=ms)

	k=0.9	k=0.8	k=0.75	k=0.7	k=0.6
heap sort	6.8 / 6.5 / 7.1 / 0.2	6.7 / 6.4 / 7.0 / 0.2	6.7 / 6.3 / 7.1 / 0.2	6.9 / 6.7 / 7.2 / 0.2	6.8 / 6.4 / 7.1 / 0.2
merge sort	6.0 / 5.7 / 6.2 / 0.2	5.9 / 5.6 / 6.1 / 0.2	6.1 / 5.7 / 6.7 / 0.3	6.5 / 6.1 / 7.2 / 0.4	6.5 / 6.0 / 7.3 / 0.4
quick sort	8.3 / 7.5 / 9.5 / 0.6	8.1 / 7.3 / 9.2 / 0.5	7.5 / 6.8 / 8.5 / 0.4	6.6 / 6.0 / 7.4 / 0.4	6.1 / 5.5 / 6.9 / 0.4
radix sort	13.1 / 12.0 / 14.5 / 0.7	13.9 / 12.8 / 15.5 / 0.8	12.9 / 11.8 / 14.3 / 0.7	12.3 / 11.2 / 13.7 / 0.7	12.5 / 11.4 / 13.9 / 0.7

충돌률 k에 대해 k가 0.75이상일 경우 다른 알고리즘들에 비해 merge sort가 유의미하게 빨랐으나 k가 0.75미만일 경우 quick sort가 가장 빨랐다. 따라서 기본값으로 quick sort를 사용하고 k가 0.75이상에서는 merge sort를 사용한다.

3) 정렬비율에 따른 알고리즘 효율성 분석

(평균/최소/최대/표준편차, 단위=ms)

	k=0.95	k=0.9	k=0.8	k=0.7
insertion sort	3.1 / 2.5 / 4.0 / 0.5	3.9 / 3.2 / 5.0 / 0.6	6.0 / 5.0 / 7.5 / 0.8	7.4 / 6.3 / 9.5 / 1.0
heap sort	5.9 / 5.5 / 6.6 / 0.3	6.2 / 5.8 / 6.9 / 0.3	6.5 / 6.1 / 7.2 / 0.3	6.6 / 6.2 / 7.4 / 0.4
merge sort	7.2 / 6.8 / 8.0 / 0.4	7.6 / 7.2 / 8.5 / 0.4	7.9 / 7.5 / 8.9 / 0.4	7.9 / 7.4 / 9.0 / 0.5
quick sort	6.4 / 5.7 / 7.5 / 0.5	6.1 / 5.5 / 7.1 / 0.5	5.9 / 5.3 / 6.8 / 0.4	5.1 / 4.6 / 5.9 / 0.3
radix sort	13.7 / 12.5 / 15.3 / 0.8	13.3 / 12.1 / 14.8 / 0.8	12.4 / 11.3 / 13.8 / 0.7	12.3 / 11.2 / 13.7 / 0.7

정렬비율 k에 대해 k에 대해 k가 0.9이상일 경우 insertion sort가 유의미 하게 빨랐으며 k가 0.9미만일 경우 평균적으로 quick sort가 가장 빨랐다. 따라서 기본값으로 quick sort를 사용하고 k가 0.9 이상에서는 insertion sort를 사용한다.

4) 입력크기가 작으면 insertion sort가 매우 효율적인 알고리즘임이 알려져 있으므로, 입력 크기가 매우 작은 ($n < 80$) 입력에 대한 insertion sort를 수행한다.

4. search 효율성 분석

data의 범위 = $(-10^9, 10^9)$ 일 경우 수행시간. 각 data 개수별로 10번 반복하여 평균값을 내었다. 2번에 의해 최고 효율의 알고리즘을 찾을 때 bubble sort는 시간이 매우 긴 비효율적인 알고리즘이므로 고려하지 않고 계산한다.

(평균/최소/최대/표준편차, 단위=ms)

	data개수= 5000	data개수=50000	data개수=500000
search알고리즘+ 알고리즘 실행	16.1 / 14.3 / 18.5 / 1.2	194.5 / 175.4 / 225.2 / 15.1	4194.2 / 3791.6 / 4801.3 / 194.7
알고리즘들의 실행시간 합	11.5 / 10.4 / 13.4 / 0.8	253.8 / 221.2 / 298.7 / 25.6	15112.6 / 14020 / 16984.5 / 803.5

실험 데이터에 의하면 data 개수가 적을때에는 search알고리즘을 이용하는 시간이 더 느렸으나 data개수가 많아지면 search 알고리즘을 이용하는 것이 다른 알고리즘들의 실행시간 합보다 빨랐다. 이론적으로 search 알고리즘은 $O(n)$ 이고 heapsort, mergesort, quicksort는 $O(n \log n)$, insertion sort는 $O(n^2)$ 으로 본 정렬 알고리즘들은 search알고리즘보다 이론적으로 더 복잡한 시간복잡도를 가지고 있으나, search알고리즘의 복잡도에서 n 앞의 계수가 매우 크기 때문에 데이터의 개수가 적을때에는 search알고리즘이 더 비효율적이고, 데이터 개수가 충분히 크다면 search 알고리즘이 더 효율적인 것이다.